

AARHUS UNIVERSITY

COMPUTER SCIENCE

NUMERICAL LINEAR ALGEBRA

Handin 9

Author

Søren M. DAMSGAARD

Student number

202309814

May 6, 2026



Zulu Bravo PsyOp Striker 9

Given a system of flowing things [1](#) with the following information:

1. A contains 300 L of mixture and 84 g of salt.
2. B contains 100 L of mixture and 36 g of salt.
3. a flows with 30 L/min of water
4. b flows with 15 L/min of mixture

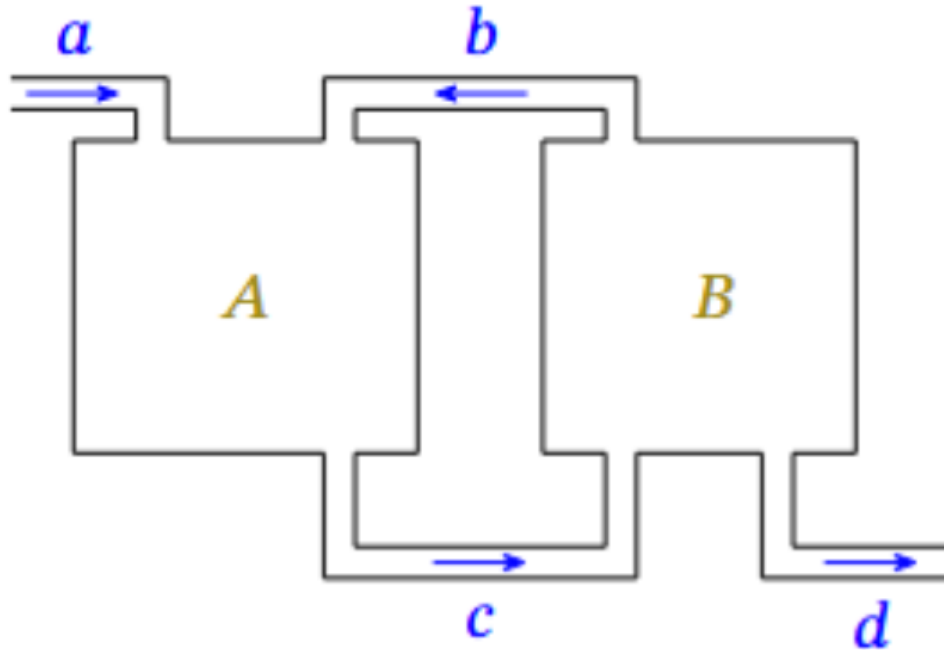


Figure 1: Containers connected by pipes: Flowchart of the system.

The Equalizer

Determining the flow of c and d so as the amount of liquid in both containers are constant.

1. A has an input a and b which totals to 45 L/min , so c needs to flow with 45 L/min to cancel out the input from a and b .
2. B has an output b and a newly determined input c , so the following should determine the flow of the remaining pipe $c - b = 30 L/\text{min} = d$

Two Peens In A System

Explain why the salt amounts $y_0(t)$ in A and $y_1(t)$ in B satisfy the system

$$\begin{aligned}y'_0(t) &= -0.15y_0(t) + 0.15y_1(t) \\ y'_1(t) &= 0.15y_0(t) - 0.45y_1(t)\end{aligned}$$

The change in salt corresponds to the input and output, percentages, of mixture in each container.

for A : c outputs 45 L/min which is 15% of A hence $-0.15y_0(t)$, and b inputs 15 L/min which is 15% of B hence $+0.15y_1(t)$.

for B : c inputs so it's $+0.15y_0(t)$ and b outputs 45 L/min which is 45% of B hence $-0.45y_1(t)$.

This give us a system that can be written as:

$$y'(t) = Ay = \begin{bmatrix} -0.15 & 0.15 \\ 0.15 & -0.45 \end{bmatrix} \begin{bmatrix} y_0(t) \\ y_1(t) \end{bmatrix}$$

with the initial value:

$$y(0) = \begin{bmatrix} 84 \\ 36 \end{bmatrix}$$

(c)Finding the EigenValues and EigenVectors as an Atheist

Given the matrix A

$$A = \begin{bmatrix} -0.15 & 0.15 \\ 0.15 & -0.45 \end{bmatrix}$$

it's characteristic polynomial is

$$\det(A - \lambda I_2) = \begin{vmatrix} -0.15 - \lambda & 0.15 \\ 0.15 & -0.45 - \lambda \end{vmatrix} = \lambda^2 + 0.60\lambda + 0.045$$

Using the quadratic formula to do stuff (find the concrete eigenvalues)

$$\lambda = \frac{-0.60 \pm \sqrt{0.60^2 - 4(1)(0.045)}}{2(1)} = \frac{-0.60 \pm \sqrt{0.18}}{2} = \frac{-0.60 \pm 0.42}{2}$$

This gives the eigenvalues:

$$\lambda_0 = -0.09 \quad \lambda_1 = -0.51 \tag{1}$$

Let's find the corresponding eigenvectors, starting with λ_0 .

$$(A - \lambda_0 I_2)v_0 = \begin{bmatrix} -0.06 & 0.15 \\ 0.15 & -0.36 \end{bmatrix} v_0 = 0$$

which gives us the eigenvector v_0 (after normalizing it).

$$v_0 = \begin{bmatrix} 0.92 \\ 0.38 \end{bmatrix}$$

Now for λ_1 .

$$(A - \lambda_1 I_2)v_1 = \begin{bmatrix} 0.36 & 0.15 \\ 0.15 & 0.06 \end{bmatrix} v_1 = 0$$

which gives us the eigenvector v_1 (after normalizing it).

$$v_1 = \begin{bmatrix} -0.38 \\ 0.92 \end{bmatrix}$$

So the eigenvectors associated with the eigenvalues are.

$$v_0 = \begin{bmatrix} 0.92 \\ 0.38 \end{bmatrix} \quad v_1 = \begin{bmatrix} -0.38 \\ 0.92 \end{bmatrix} \quad (2)$$

The Author argues that since god is dead, no one will judge the lack of intermediates shown for the eigenvectors...please? #PuppyEyes

Solutions for $y_0(t)$ and $y_1(t)$

Finding a solution for $y'_0(t)$ and $y'_1(t)$ using the eigen thingies, proposition 22.3 [1] says a solution is given using the eigenvalues and eigenvectors as such:

$$y(t) = c_0 v_0 e^{\lambda_0 t} + c_1 v_1 e^{\lambda_1 t}$$

Inserting the values from (1) and (2).

$$y(t) = c_0 \begin{bmatrix} 0.92 \\ 0.38 \end{bmatrix} e^{-0.09t} + c_1 \begin{bmatrix} -0.38 \\ 0.92 \end{bmatrix} e^{-0.51t} \quad (3)$$

To isolate and determine c_0 and c_1 , the initial state for $y(0)$ can be used.

$$y(0) = \begin{bmatrix} 84 \\ 36 \end{bmatrix} = c_0 \begin{bmatrix} 0.92 \\ 0.38 \end{bmatrix} + c_1 \begin{bmatrix} -0.38 \\ 0.92 \end{bmatrix} = \begin{bmatrix} c_0 \cdot 0.92 + c_1 \cdot -0.38 \\ c_0 \cdot 0.38 + c_1 \cdot 0.92 \end{bmatrix}$$

This gives us the following system:

$$0.92c_0 - 0.38c_1 = 84$$

$$0.38c_0 + 0.92c_1 = 36$$

Using the god-given gift of numpy:

```
1 B = np.array([[0.92, -0.38],
2               [0.38, 0.92]])
3
4 b = np.array([84, 36])
5
6 c0, c1 = np.linalg.solve(A, b)
7
8 print('c0 = ', c0)
9 print('c1 = ', c1)
10
11 # Output for reference (not part of the actual code)
12 c0 = 91.80460234154218
13 c1 = 1.2111425111021366
```

Rounding down c_0 and c_1

$$c_0 = 91.80 \quad c_1 = 1.21$$

Now it's possible to determine a solution for both $y_0(t)$ and $y_1(t)$, by using the newfound constants and plotting them into (3)

$$y(t) = 91.80 \begin{bmatrix} 0.92 \\ 0.38 \end{bmatrix} e^{-0.09t} + 1.21 \begin{bmatrix} -0.38 \\ 0.92 \end{bmatrix} e^{-0.51t} = \begin{bmatrix} 84.456e^{-0.09t} - 0.4598e^{-0.51t} \\ 34.884e^{-0.09t} + 1.1132e^{-0.51t} \end{bmatrix}$$

Plotting

Plotting both y_0 and y_1 versus time, shows an exponential decrease in salt over time which is expected since the biggest term -0.45 represents the output in container B from pipe d .

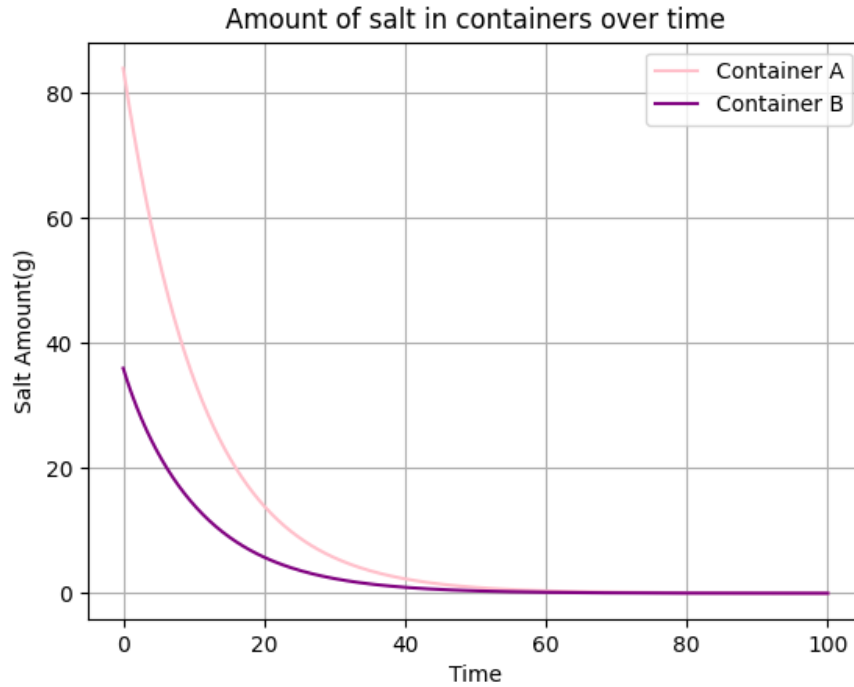


Figure 2

For the y_0 versus y_1 it shows the relation in how the containers salt levels are related.

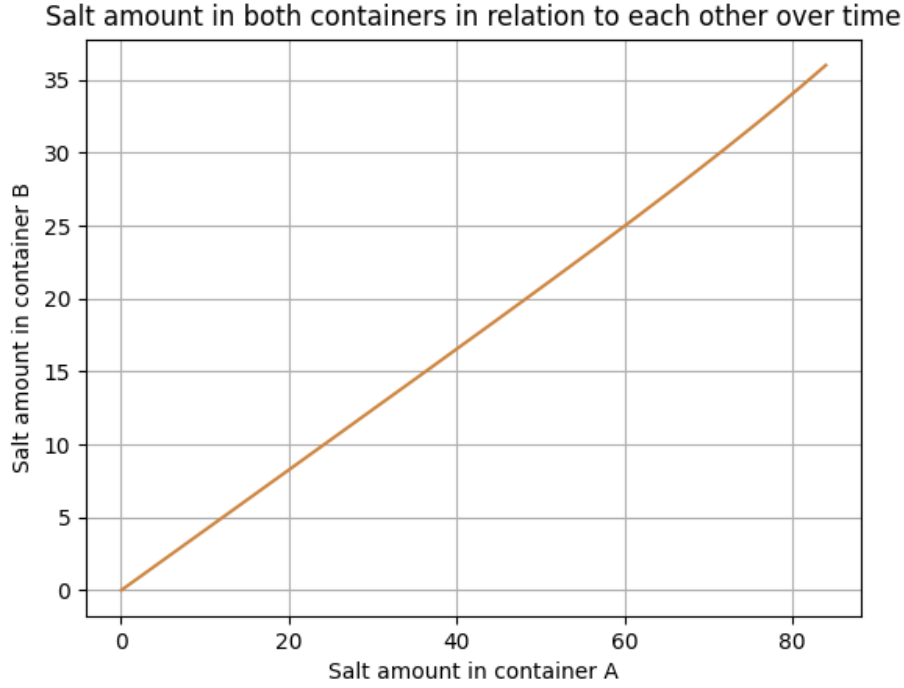


Figure 3

This specifically shows a linear relation in the salt levels, since both container A and B drain the same rate over time.

Limits of ∞

Taking (3) and splitting it into y_0 and y_1

$$\begin{aligned} y_0(t) &= c_0 0.92e^{-0.09t} - c_1 0.38e^{-0.51t} \\ y_1(t) &= c_0 0.38e^{-0.09t} + c_1 0.92e^{-0.51t} \end{aligned}$$

it's know that $e^{-t} \rightarrow 0$ as $t \rightarrow \infty$ so the solution will approach zero as t grows, which makes sense in regards to 2 and 3.

It also means that the terms that decay the fastest can be seen as negligible, so any terms with $e^{-0.51t}$

$$\lim_{t \rightarrow \infty} \frac{y_1(t)}{y_0(t)} = \frac{c_0 0.38e^{-0.09t}}{c_0 0.92e^{-0.09t}} = \frac{0.38}{0.92} \approx 0.41 \quad (4)$$

Which is the ratio between the salt level in A and B as t changes.

Appendix

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

```

3
4 # Finding Eigenvalues/vectors (c)
5 A = np.array([[ -0.15,  0.15],
6               [ 0.15, -0.45]])
7 # Please don't end me... \(;_;/
8 eigvalues, eigvectors = np.linalg.eig(A)
9
10
11 # solving for the constants c_0 and c_1 in (d)
12 B = np.array([[ 0.92, -0.38],
13               [ 0.38,  0.92]])
14
15     b = np.array([84, 36])
16
17 c0, c1 = np.linalg.solve(B, b)
18
19
20 # (e)
21 t = np.linspace(0,100,1000)
22 y_0 = 84.456 * np.e**(-0.09*t) - 0.4598 * np.e**(-0.51*t)
23 y_1 = 34.884 * np.e**(-0.09*t) + 1.1132 * np.e**(-0.51*t)
24
25
26 ## Brr brrr printing section
27 print(f'Eig Values:\n {eigvalues}')
28 print(f'Eig Vectors:\n {eigvectors}')
29 print('c0 = ', c0)
30 print('c1 = ', c1)
31
32 fig1, y = plt.subplots()
33 y.plot(t,y_0, color='pink', label='Container A')
34 y.plot(t, y_1, color='purple', label='Container B')
35 y.set_title('Amount of salt in containers over time')
36 y.set_xlabel('Time')
37 y.set_ylabel('Salt Amount(g)')
38 y.grid()
39 y.legend()
40
41 fig2, pl = plt.subplots()
42 pl.plot(y_0,y_1, color='peru')
43 pl.set_xlabel('Salt amount in container A')
44 pl.set_ylabel('Salt amount in container B')
45 pl.grid()
46 pl.set_title('Salt amount in both containers in relation to each other
47               over time')
48 plt.show()

```

References

- [1] Andrew Swan. *Numerisk Linær Algebra*. Institut for Matematik, Aarhus Universitet, 2025.